

0) UserRegistration : le role de l'admin

1) Connexion utilisateur

- L'utilisateur (patient, docteur, assureur) se connecte avec **MetaMask**.
- Le frontend récupère `msg.sender` via Ethers.js : l'adresse Ethereum de l'utilisateur.
- Le smart contract `AccessControl` sait automatiquement **qui est patient, docteur ou assureur** grâce à `roles[msg.sender]`.

2) Dépôt de dossiers médicaux (pour le docteur)

1. Le docteur téléverse un fichier médical (PDF, image...) sur **Pinata/IPFS** via le backend.
2. Le backend calcule le **hash du fichier** et crée un JSON de métadonnées (patient, docteur, type de document, CID IPFS...).
3. Le backend appelle la fonction `addRecord(patient, ipfsHash, fileHash, recordType)` de `MedicalRecord.sol`.
4. Le smart contract enregistre le dossier et émet l'événement `MedicalRecordAdded`.

Important :

- Le patient peut consulter ce dossier mais **ne peut pas le créer**.
- L'assureur peut aussi accéder pour vérifier la validité lors d'une réclamation.

3) Création de polices d'assurance (pour l'assureur)

1. L'assureur crée une police pour un patient via le frontend.
2. Il indique le montant de couverture, le taux de remboursement, la durée, et le type de soins.
3. Le backend appelle `createPolicy(patient, coverageAmount, coverageRate, duration, careType)` de `PolicyContract.sol`.
4. La police est maintenant **active** et liée au patient et à l'assureur.

4) Soumission d'une réclamation (pour le patient)

1. Le patient choisit un **dossier médical** et la police correspondante dans le frontend.
2. Le patient soumet le montant à réclamer.
3. Le backend appelle `submitClaim(policyId, recordId, amountRequested)` dans `ClaimContract.sol`.
4. Le smart contract calcule le montant remboursable (`_calculateReimbursement`) et enregistre la claim comme **Pending**.

5. Un événement `ClaimSubmitted` est émis.

5) Soumission d'une réclamation (pour le patient)

1. Le patient choisit un **dossier médical** et la police correspondante dans le frontend.
2. Le patient soumet le montant à réclamer.
3. Le backend appelle `submitClaim(policyId, recordId, amountRequested)` dans `ClaimContract.sol`.
4. Le smart contract calcule le montant remboursable (`_calculateReimbursement`) et enregistre la claim comme **Pending**.
5. Un événement `ClaimSubmitted` est émis.

6) Accès aux données et audit

- Le patient peut consulter **ses dossiers et claims** via `getPatientRecords` et `getPatientClaims`.
- L'assureur peut consulter les dossiers et claims de ses patients.
- L'admin peut tout vérifier et révoquer des dossiers si nécessaire.
- Tous les événements sur la blockchain assurent **transparence et traçabilité**.

=> ce qui manque :

- Page d'enregistrement des utilisateurs → Admin enregistre Insurer, Doctor, Patient dans `AccessControl`
- Vérification du rôle connecté → Actuellement n'importe qui voit tout → Il faut détecter le rôle via `accessControl.checkRole(address)` → Et afficher le bon dashboard selon le rôle
- côté assureur: Créer une police (`createPolicy`) / Approuver/Rejeter un CLAIM (Actuellement il approuve un dossier médical — c'est faux !) / Voir les claims en attente
- Côté Patient: Voir ses polices d'assurance / Soumettre un claim (`submitClaim`) /
- Côté Admin: Dashboard admin complet → Enregistrer/désactiver des utilisateurs → `accessControl.registerUser()` → `accessControl.setUserActive()` → `accessControl.changeUserRole()` / Révoquer un dossier médical → `MedicalRecord.revokeRecord()`

